



Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №7
З дисципліни «Технології розроблення програмного забезпечення»
Тема: «**ШАБЛОН «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE
METHOD»»**
Система ведення нотаток

Виконав:
Студент групи ІА-24
Шкарніков А. С.

Перевірив:
Мягкий М. Ю.

Київ-2024

Зміст

Тема:.....	3
Мета:	3
Хід роботи	3
1. Реалізувати не менше 3-х класів відповідно до обраної теми	3
2. Реалізувати один з розглянутих шаблонів за обраною темою	5
3. Перевірка патерну	7
Висновки:	7
Код проекту:	8

Тема:

ШАБЛОН «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD»

Мета:

Ознайомитися з основними шаблонами проектування, такими як «Mediator», «Facade», «Bridge», «Template Method», дослідити їхні принципи роботи та навчитися використовувати для створення гнучкого та масштабованого програмного забезпечення, зокрема у розробці музичного програвача.

Завдання:

В якості нотаток має бути, як мінімум, текст або зображення, додатково можна додати підтримку файлів розміром до 2Мб, збереження посилань на веб-сторінки, збереження html-тексту з коректним його відображенням. Система повинна дозволяти вести нотатки онлайн, заводити блокноти, прикріплювати до нотаток мітки, давати можливість працювати в системі одночасно багатьом користувачам і одному користувачу працювати з різних комп'ютерів. Користувач повинен мати можливість дати доступ до свого блокноту з замітками іншому користувачу з різними правами (тільки перегляд; перегляд та редагування; перегляд, редагування, додавання та видалення нотаток)

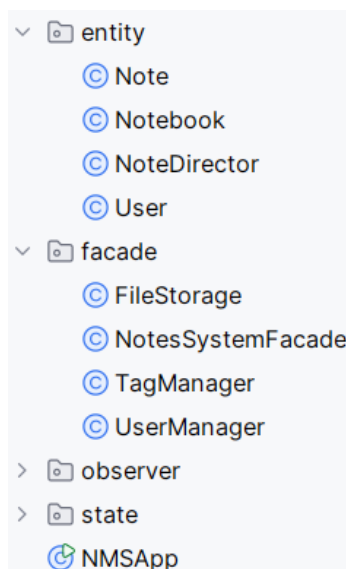
Хід роботи**1. Реалізувати не менше 3-х класів відповідно до обраної теми**

Рис. 1 — Структура проекту

У процесі виконання лабораторної роботи було реалізовано 4 класи, які забезпечують реалізацію функціоналу музичного програвача (рис. 1). Наведемо детальний опис кожного з цих класів:

1. AudioPlayerLibrary

Це низькорівневий клас, який відповідає за взаємодію з бібліотекою для відтворення аудіо. Він надає базові функціональні можливості для завантаження джерела аудіо (локальний файл або потік), запуску та зупинки відтворення музики. При цьому інкапсулює всі операції, пов'язані з аудіо, і забезпечує прямий доступ до функціоналу бібліотеки.

2. UIRendererLibrary

Клас для роботи з елементами інтерфейсу користувача. Він включає методи для оновлення інформації про поточний трек (назва, прогрес відтворення) та роботи з візуальними елементами програвача, наприклад, прогрес-баром. Забезпечує керування візуальним представленням музичного програвача і його інтерактивними компонентами.

3. StreamingService

Клас для завантаження та роботи з потоковою музикою. Він містить функції для підключення до віддаленого потоку через URL, буферизації поточних даних, зміни джерела потоку для відтворення. При цьому інкапсулює всі процеси, пов'язані з потоковою передачею даних, забезпечуючи простий інтерфейс для роботи з потоковою музикою.

4. MediaFacade

Клас-фасад, який об'єднує роботу трьох попередніх класів (AudioPlayerLibrary, UIRendererLibrary, StreamingService) і надає простий у використанні API. Завдяки фасаді, клієнтський код може запускати відтворення локальних треків або потокової музики, зупиняти відтворення та оновлювати UI програвача (назва треку, прогрес відтворення). Таким чином абстрагує складність взаємодії з низькорівневими бібліотеками, спрощує клієнтський код і зменшує залежності, забезпечуючи єдиний інтерфейс для роботи з медіа.

2. Реалізувати один з розглянутих шаблонів за обраною темою



Рис. 2 — Діаграма класів

Патерн Facade реалізований через клас **NotesSystemFacade**, який виступає як єдина точка входу до всієї системи нотаток. Цей фасад інкапсулює взаємодію між різними підсистемами (управління користувачами, файлове сховище, система тегів) та надає простий, уніфікований інтерфейс високого рівня для роботи з системою. Замість того, щоб клієнтський код взаємодівав з кожною підсистемою окремо, він працює лише з фасадом, який координує роботу всіх компонентів.

Реалізація

- **AudioPlayerLibrary** надає функціонал для роботи з аудіо (завантаження файлів, запуск/зупинка відтворення).
- **UIRendererLibrary** відповідає за оновлення елементів інтерфейсу (назва треку, прогрес відтворення).
- **StreamingService** забезпечує завантаження і обробку потокових джерел музики.

Клієнтський код не взаємодіє безпосередньо з цими класами, а використовує **MediaFacade**. Facade отримує виклики від клієнта, розподіляє їх між відповідними підсистемами та координує їхню роботу.

Проблеми, які вирішує патерн

- **Складність інтеграції** - фасад приховує складну логіку взаємодії між компонентами системи, надаючи простий інтерфейс для типових операцій.
- **Зв'язаність коду** - зменшується зв'язаність між клієнтським кодом та підсистемами, оскільки клієнт працює тільки з фасадом.

- Дублювання коду - фасад централізує логіку перевірки прав доступу та послідовність операцій, запобігаючи дублюванню цієї логіки в різних частинах системи.
- Складність управління безпекою - фасад забезпечує централізований контроль доступу до всіх операцій.

Переваги використання

- Спрощений інтерфейс - клієнти отримують простий, зрозумілий інтерфейс для роботи з складною системою.
- Покращена підтримка - зміни в підсистемах можуть бути зроблені без впливу на клієнтський код.
- Централізований контроль - всі операції проходять через єдину точку входу, що спрощує логування, моніторинг та контроль безпеки.
- Гнучкість розширення - нові функції можуть бути додані до фасаду без змін в існуючому клієнтському коді.
- Зручність тестування - фасад надає чіткий інтерфейс для написання тестів.

В контексті системи нотаток патерн Facade особливо корисний, оскільки система має багато взаємопов'язаних функцій (створення нотаток, управління файлами, теги, права доступу). Фасад дозволяє клієнтам легко виконувати складні операції через прості виклики методів, не турбуючись про внутрішню реалізацію. Наприклад, при створенні нової нотатки з файлом фасад автоматично координує роботу з файловим сховищем, створення запису про нотатку та встановлення прав доступу - все це через один простий метод.

3. Перевірка патерну

```
public class NMSApp {  
    public static void main(String[] args) {  
        NotesSystemFacade noteSystem = new NotesSystemFacade();  
  
        // Створення нової текстової нотатки  
        String noteId = noteSystem.createNote( userId: "user1", content: "Важлива нотатка", type: "text");  
  
        // Додавання тегу  
        noteSystem.addTagToNote( userId: "user1", noteId, tag: "важливе");  
  
        // Надання доступу іншому користувачу  
        noteSystem.shareNote( ownerId: "user1", targetUserId: "user2", noteId, permission: "view");  
  
        // Додавання файлу  
        byte[] fileContent = "Тестовий контент файлу".getBytes();  
        noteSystem.addFileToNote( userId: "user1", noteId, fileContent);  
    }  
}
```

Рис. 3 — Перевірка роботи

У методі main демонструється використання патерна Observer. У першій частині коду створюється новий екземпляр фасаду NotesSystemFacade, який надає доступ до всіх функцій системи нотаток. Після цього створюється нова текстова нотатка для користувача "user1" з текстом "Важлива нотатка", в результаті чого система генерує та повертає унікальний ідентифікатор нотатки (noteId). Далі до цієї нотатки додається тег "важливе", що дозволить пізніше легко знаходити цю нотатку серед інших.

```
Створено нову нотатку: a0871c37-391e-451d-a11b-fa01971306fe  
Додано тег 'важливе' до нотатки a0871c37-391e-451d-a11b-fa01971306fe  
Надано доступ користувачу user2 до нотатки a0871c37-391e-451d-a11b-fa01971306fe з правами: view  
Збережено файл: bff2da64-0754-461a-91d3-a8d248d9954f  
Додано файл до нотатки: a0871c37-391e-451d-a11b-fa01971306fe
```

Рис. 4 — Результат роботи

Далі виконуються операції з налаштування доступу та додавання вмісту до створеної нотатки. Спочатку надається доступ користувачу "user2" з правами лише на перегляд нотатки (параметр "view"). Після цього створюється тестовий контент файлу у вигляді масиву байтів і цей файл прикріплюється до нотатки. Всі ці операції виконуються через єдиний інтерфейс фасаду, який приховує складну внутрішню логіку взаємодії різних компонентів системи.

Висновки:

У ході лабораторної роботи ми реалізували патерн Facade у контексті системи ведення нотаток. Цей підхід дозволив створити єдиний інтерфейс для взаємодії з

різними підсистемами, включаючи управління користувачами, зберігання файлів, роботу з тегами та керування нотатками. Завдяки використанню фасаду ми значно спростили складну архітектуру системи, централізували контроль доступу, зменшили зв'язаність між компонентами та забезпечили простий і зрозумілий спосіб роботи з системою для клієнтського коду. Реалізація цього патерну зробила систему більш гнучкою для майбутніх розширень та полегшила її підтримку.

Код проекту:

https://github.com/Atl4sDev/TRPZ_labs